

Programa de Pós-Graduação em Computação Aplicada
Universidade de Brasília

Geração Automatizada de Testes Baseada em Model Checking: Um Estudo Aplicado ao e-Título Versão para Qualificação

Thiago Viana Fernandes
thiagu.vf@gmail.com

22 de fevereiro de 2026

- ▶ APIs REST são base de sistemas modernos e críticos.
- ▶ Erros de integração frequentemente não são detectados por testes tradicionais.
- ▶ Testes manuais e exploratórios possuem baixa sistematização.
- ▶ Há lacuna entre modelos formais e práticas de teste de software.

- ▶ Como gerar testes de integração de forma sistemática e baseada em especificação?
- ▶ Como garantir coerência entre regras de negócio, serviços REST e testes?
- ▶ Como explorar contra-exemplos de Model Checking além da verificação?

- ▶ Modelar comportamento do sistema.
- ▶ Verificar propriedades com Model Checking.
- ▶ Usar contra-exemplos como fonte de contratos e testes.

- ▶ Técnica formal para verificação automática de propriedades.
- ▶ Baseada em exploração exaustiva de estados.
- ▶ Propriedades expressas em LTL.
- ▶ Ferramenta utilizada: SPIN.

- ▶ Pré-condições.



- ▶ Pós-condições.



- ▶ Invariantes.



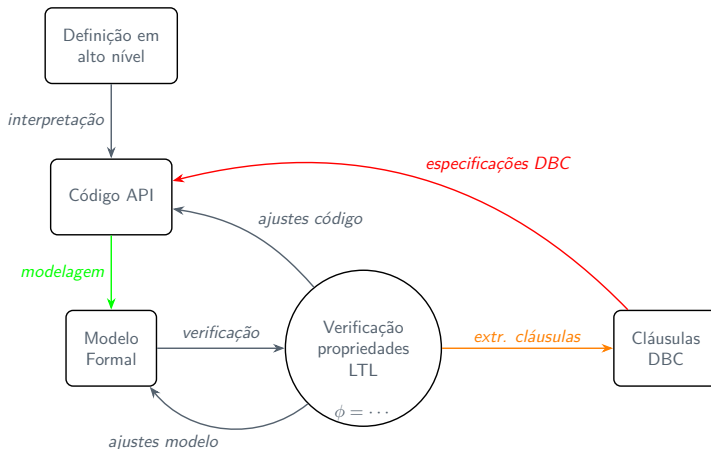
- ▶ Contratos como oráculos de teste.

- ▶ Validação de contratos da API.
- ▶ Sequências de chamadas.
- ▶ Estados intermediários.
- ▶ OpenAPI como suporte à especificação.

- ▶ Modelagem comportamental em Promela.
- ▶ Definição de propriedades LTL.
- ▶ Execução do Model Checker.
- ▶ Extração de contra-exemplos.
- ▶ Geração de contratos e testes.

Método Proposto

Visão Geral



- ▶ Processos representando serviços.
- ▶ Estados explícitos.
- ▶ Comunicação por variáveis globais.

- ▶ Especificam comportamentos esperados.
- ▶ Exemplo: após carrossel válido, senha deve ser definida.

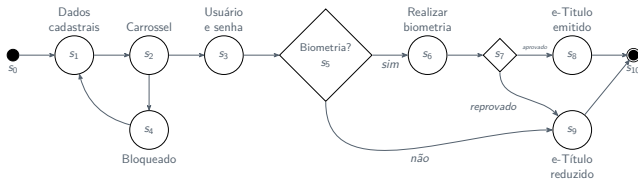
`[] (carrossel_ok -> <> senha_definida)`

- ▶ Caminho que viola propriedade.
- ▶ Sequência concreta de estados.
- ▶ Evidência de falha lógica.

- ▶ Estados iniciais $\rightarrow$ pré-condições.
- ▶ Estado final inválido $\rightarrow$ pós-condição.
- ▶ Regras como cláusulas formais.

- ▶ Pré-condições → dados de entrada.
- ▶ Sequência → passos do teste.
- ▶ Pós-condições → asserções.

► Fluxo de onboarding.



- Carrossel, senha e biometria.
- Sistema real e crítico.

- ▶ Propriedades violadas identificadas.
- ▶ Contratos derivados automaticamente.
- ▶ Casos de teste obtidos.
- ▶ Detecção de cenários não cobertos.

- ▶ Proposta de integração entre Model Checking, Design by Contract e testes de APIs REST.
- ▶ Demonstração de que contra-exemplos podem ser sistematicamente convertidos em cláusulas contratuais e casos de teste.
- ▶ Evidência de que a abordagem auxilia na identificação de falhas lógicas e de uso indevido de serviços.
- ▶ Validação em um fluxo real do aplicativo e-Título.

Obrigado!